

基于 llama.cpp 的本地 RAG 问答系统

一、项目背景与目标

大语言模型（Large Language Model，LLM）正在深刻改变人机交互方式。然而，直接调用云端 API 存在隐私风险、网络延迟和成本问题。检索增强生成（Retrieval-Augmented Generation，RAG）是一种将向量检索与本地推理相结合的架构，能够让模型基于特定文档进行精准问答，而无需将私有数据上传至外部服务器。

本大作业要求同学们以 llama.cpp 的 C API 为推理后端，从零构建一个完整的本地 RAG 问答系统，覆盖文档处理、向量嵌入、向量检索和大模型生成四大核心模块，最终实现可运行的命令行问答程序。

学习目标：综合运用 C++ 类设计、RAII 资源管理、STL 容器与算法、文件 I/O、多态与继承、UTF-8 字符串处理等核心知识，并接触真实工程级 C 库的封装与集成。

二、系统架构要求

系统须按照以下模块划分进行设计，各模块以独立的 .h / .cc 文件实现：

模块名称	对应文件	核心职责
配置管理	config.h / config.cc	从 JSON 文件加载运行参数，提供合理默认值
文档处理	document.h / document.cc	读取 TXT 小说，按章节切分为 Chunk 列表
数据结构	chunk.h / chunk.cc	定义 Chunk 结构体及二进制序列化/反序列化
向量数据库	vectordatabase.h / vectordatabase.cc	存储向量、余弦检索、持久化到磁盘
向量引擎	embeddingengine.h / embeddingengine.cc	调用 Embedding 模型将文本转为向量
生成引擎	generationengine.h / generationengine.cc	调用生成模型进行流式推理输出

模块名称	对应文件	核心职责
模型基类	llamamodel.h / llamamodel.cc	统一管理 llama_model / llama_context 生命周期
应用主体	mrageapp.h / mrageapp.cc	协调各模块完成建库与问答两大流程
程序入口	main.cc	解析命令行参数，选择运行模式

三、功能需求详述

3.1 配置管理模块（AppConfig）

AppConfig 结构体须包含以下字段，支持从 config.json 读取，字段缺失时保留编码默认值：

- **模型路径：** emb_model_path （嵌入模型）， gen_model_path （生成模型）
- **GPU 层数：** n_gpu_layers （默认 99，即全量 GPU 加载）
- **文档切分：** chunk_size （默认 500 字节）， overlap_size （默认 50 字节）
- **检索参数：** top_k （默认返回 3 个片段）
- **生成模型上下文：** gen_n_ctx 、 gen_n_batch 、 gen_n_ubatch 、 max_output_tokens
- **采样参数：** temperature 、 top_k_sampler 、 top_p
- **嵌入模型上下文：** emb_n_ctx 、 emb_n_batch 、 emb_n_ubatch

要求： 使用 nlhmann/json 解析配置文件。若文件不存在，不报错，全部使用默认值。

示例 config.json：

```
{
  "models": {
    "embedding":  "./models/bge-small-zh-v1.5-f16.gguf",
    "generation":  "./models/qwen2.5-1.5b-instruct-q4_k_m.gguf",
    "n_gpu_layers": 99
  },
  "document": { "chunk_size": 500, "overlap_size": 50 },
  "retrieval": { "top_k": 3 },
  "generation": {
    "n_ctx": 4096, "n_batch": 4096, "n_ubatch": 512,
    "max_output_tokens": 512,
    "temperature": 0.7, "top_k": 40, "top_p": 0.9
  },
  "embedding": { "n_ctx": 512, "n_batch": 512, "n_ubatch": 512 }
}
```

3.2 文档处理模块（DocumentProcessor）

`processNovel(filepath)` 方法须完成以下工作：

- 1. 逐行读取 UTF-8 编码的 TXT 文件，清除 Windows 换行符（`\r`）
- 2. 识别章节标题（正则：`^(第.*[章节回卷]|Chapter\s+\d+)`），以章节名作为 `metadata` 打标签
- 3. 按 `chunk_size` 字节阈值切分文本，保证在 UTF-8 字符边界处切割，不截断多字节字符
- 4. 实现 **overlap 机制**：新块开头保留上一块末尾 `overlap_size` 字节的内容，切割点优先对齐到中文句末标点（`。！？`）或换行符
- 5. 文件末尾残余文本（未达到阈值）须作为最后一个 `Chunk` 保存

注意： 所有切割必须严格保证 UTF-8 合法性。建议实现 `nextUtf8Boundary` 工具函数辅助处理。

3.3 数据结构模块（Chunk）

`Chunk` 须为可高效移动的结构体，包含以下字段：

字段	类型	说明
<code>id</code>	<code>size_t</code>	全局唯一递增 ID
<code>text</code>	<code>string</code>	文本内容
<code>metadata</code>	<code>string</code>	所属章节名
<code>embedding</code>	<code>vector<float></code>	向量，建库阶段填充

须实现二进制序列化/反序列化接口：

```
void serialize(std::ofstream &out) const;
static Chunk deserialize(std::ifstream &in);
```

格式规范： 写入顺序为 `id` → `text`长度+内容 → `metadata`长度+内容 → `embedding`维度+浮点数组。 `size_t` 字段建议统一为 `uint64_t` 以保证跨平台兼容性。

3.4 向量数据库模块（VectorDatabase）

须实现以下核心功能：

- 1. `insert()`：批量或单条插入 `Chunk`

2. `search(query_emb, top_k)`：返回余弦相似度最高的 `top_k` 个 Chunk，使用**最小堆**（`priority_queue`）实现 $O(n \log k)$ 检索
3. `saveToDisk(path)`：二进制序列化写盘，文件头包含魔数（`0x4D524147`）和版本号
4. `loadFromDisk(path)`：读取并校验文件头，失败时返回 `false` 并输出错误信息

余弦相似度公式：

$$\text{cosine}(A, B) = \frac{A \cdot B}{\|A\| \cdot \|B\|}$$

3.5 模型基类（LlamaModelBase）

须遵循 **RAII 原则** 管理 `llama_model*` 与 `llama_context*` 的完整生命周期：

- **构造函数**：根据 `ModelMode`（`Embedding / Generation`）加载对应模型，创建上下文
- **析构函数**：先释放 `ctx_`，再释放 `model_`，顺序不可颠倒
- **禁用拷贝构造和赋值运算符**（Rule of Five）
- 提供 `protected tokenize()` 工具方法，两次调用 `llama_tokenize` 以获取精确 token 数

关键区别：Embedding 模型须设置

`ctx_params.embeddings = true`，`pooling_type = LLAMA_POOLING_TYPE_MEAN`；生成模型须设置 `embeddings = false`。两者上下文参数互相独立，分别来自 `AppConfig` 的 `emb_*` 和 `gen_*` 字段。

3.6 向量引擎（EmbeddingEngine）

`generateEmbedding(text)` 须按以下步骤执行：

1. 清理输入文本中残缺的 UTF-8 序列（`sanitizeUtf8`）
2. 调用 `tokenize()`，截断至 `min(n_ctx, n_batch)` 上限
3. 调用 `llama_memory_clear` 清空 KV Cache
4. 手动构造 `llama_batch`，最后一个 token 的 logits 设为 1
5. 调用 `llama_decode` 执行推理
6. 优先通过 `llama_get_embeddings_seq` 获取序列向量，失败时回退到 `llama_get_embeddings`
7. 返回长度为 `llama_model_n_embd(model_)` 的 float 向量

3.7 生成引擎 (GenerationEngine)

generateStream(query, chunks) 须完成完整的 RAG 推理流程：

1. 清空 KV Cache
2. 将 chunks 格式化为带出处标注的上下文字符串
3. 组装 **ChatML 格式** 的 Prompt (system + user + assistant 起始标记)
4. Prompt Prefill: 将全部 tokens 装入 batch, 仅最后一个 token 开启 logits
5. 调用 llama_decode 完成 prefill
6. 初始化采样链, **顺序不可颠倒**: top_k → top_p → temperature → dist
7. 自回归生成循环: 采样 → 检测 EOG → 输出 → 清空 batch → 加入新 token → decode
8. 释放 sampler 与 batch

Prompt 模板格式：

```
<|im_start|>system
```

你是一个严谨的小说阅读助手。请你【必须并且只能】根据下面提供的[参考上下文]来回答用户的问题。

如果上下文中有答案，请详细回答并引用出处；如果上下文中没有提到相关信息，请直接回答"根据当前片段无法得

```
<|im_start|>user
```

[参考上下文]：

```
{context}
```

用户问题: {query}<|im_end|>

```
<|im_start|>assistant
```

3.8 应用主体 (MragApp)

须包含以下设计要点：

- **BackendGuard 内部结构体**：在成员列表最前声明，利用 C++ 构造/析构顺序保证 llama_backend_init / llama_backend_free 正确包裹所有引擎
- **buildKnowledgeBase(txt, db)**：切分 → 逐块嵌入 → 插入数据库 → 持久化
- **chatLoop(db)**：加载数据库 → 循环读取问题 → 嵌入问题 → 检索 → 生成回答
- **AppConfig 以值存储**（非引用），避免悬空引用风险

3.9 命令行接口 (main.cc)

程序须支持以下两种运行模式：

```
# 离线建库（只需执行一次）
./mrag --ingest <小说TXT路径> <输出数据库路径>

# 在线问答
./rag --chat <数据库路径>
```

须通过 `llama_log_set` 注册空回调，屏蔽 `llama.cpp` 的底层日志，只保留自定义进度信息。

四、非功能需求

4.1 编译与环境

- 使用 **CMake** 构建，C++ 标准：**C++17** 或以上
- 依赖项：`llama.cpp`（CMake `FetchContent` 自动拉取）、`nlohmann/json`（header-only）
- 须在 Linux x86-64 环境下通过编译，推荐 Ubuntu 22.04 / 24.04
- `CMakeLists.txt` 须正确链接 `llama` 和 `ggml`

4.2 内存安全

- 所有 `llama C API` 资源须通过 `RAII` 类管理，禁止裸露的 `new/delete` 与 `C API` 混用
- `llama_batch` 须在使用后通过 `llama_batch_free` 释放
- `llama_sampler_chain` 须通过 `llama_sampler_free` 释放
- 不得出现内存泄漏（可使用 `valgrind` 验证，详见加分项）

4.3 错误处理

- 模型文件不存在时，抛出 `std::runtime_error` 并携带可读路径信息
- `llama_decode` 返回非零时，输出错误信息并安全退出当前推理循环
- 数据库文件魔数/版本号不匹配时，拒绝加载并提示原因
- 输入 Prompt 超出上下文窗口时，输出警告并跳过本次推理

4.4 代码质量

- 每个 `.h` 文件须有 `#pragma once` 或 `include guard`
- 禁止在头文件中定义非模板函数体（防止重复定义链接错误）
- 类成员函数须区分 `const` / `non-const`
- 关键函数须有不超过 3 行的中文注释说明其用途

五、评分标准

5.1 基础分（100 分）

评分维度	细分项	分值	验收标准
功能正确性（40 分）	建库流程正常运行	10	能成功切分小说并生成数据库文件
	问答流程正常运行	10	能加载数据库并进行多轮问答
	向量检索结果合理	10	检索到的片段与问题语义相关
	大模型输出连贯	10	回答流式输出，无乱码，能引用上下文
代码设计（30 分）	RAII 资源管理	10	LlamaModelBase 析构正确，无资源泄漏
	模块划分与接口设计	10	各模块职责清晰，头文件接口最小化
	错误处理完整性	10	边界情况有防御，异常携带可读信息
文档报告（20 分）	README 使用说明	8	含编译步骤、模型下载、运行命令示例
	设计说明文档	12	含架构图、各类职责说明、已知局限性
编译与运行（10 分）	零修改编译通过	5	cmake + make 无报错
	两种模式均可运行	5	--ingest 和 --chat 均能正常使用

5.2 加分项（最多 +10 分）

加分项	分值	说明
Valgrind 无内存泄漏	+3	提供 valgrind --leak-check=full 输出截图
支持配置文件热切换	+3	运行时不重启即可切换模型路径
多文档建库支持	+4	--ingest 模式可一次处理多个 TXT 文件

 **重要提示：** 代码相似度超过 30%（与同学或 AI 生成内容高度雷同）将直接判定为 **0 分**，并上报学院学术诚信委员会。最终提交须附个人承诺书。

六、提交规范

6.1 目录结构

压缩包解压后须符合以下结构（学号 替换为本人学号）：

```
<学号>_mrag/  
├─ CMakeLists.txt  
├─ README.md  
├─ DESIGN.md          # 设计说明文档  
├─ config.json        # 示例配置文件  
└─ src/  
    ├─ main.cc  
    ├─ config.h  
    ├─ config.cc  
    ├─ chunk.h  
    ├─ chunk.cc  
    ├─ document.h  
    ├─ document.cc  
    ├─ vectordatabase.h  
    ├─ vectordatabase.cc  
    ├─ llamamodel.h  
    ├─ llamamodel.cc  
    ├─ embeddingengine.h  
    ├─ embeddingengine.cc  
    ├─ generationengine.h  
    └─ generationengine.cc
```

6.2 README 须包含内容

1. 依赖项安装命令（apt / cmake）
2. 模型文件下载地址及放置路径
3. 编译命令示例
4. 运行示例（--ingest 和 --chat 两个模式各一例）
5. 已知问题或局限性说明

6.3 提交方式

请将压缩包上传至课程管理平台作业区，文件命名格式：

<学号>_<姓名>_mrag.zip



截止时间： 第 16 周周五 23:59。逾期每天扣 10 分，逾期 3 天后不再接受提交。

七、参考资源

7.1 推荐模型

用途	推荐模型	下载来源	显存占用
文本嵌入	bge-small-zh-v1.5-f16.gguf	Hugging Face / ModelScope	< 200 MB
文本生成	qwen2.5-1.5b-instruct-q4_k_m.gguf	Hugging Face / ModelScope	约 1 GB
文本生成 (高配)	qwen2.5-7b-instruct-q4_k_m.gguf	Hugging Face / ModelScope	约 4.5 GB

7.2 参考文档

- llama.cpp 官方仓库：<https://github.com/ggml-org/llama.cpp>
- llama.cpp C API 头文件：include/llama.h （重点阅读 batch、context、sampler 相关函数）
- nlohmann/json 文档：<https://json.nlohmann.me>
- BGE 模型说明：<https://huggingface.co/BAAI/bge-small-zh-v1.5>
- Qwen2.5 模型说明：<https://huggingface.co/Qwen/Qwen2.5-1.5B-Instruct>

7.3 关键 API 速查

API 函数	用途说明
llama_model_load_from_file(path, params)	从 GGUF 文件加载模型权重
llama_init_from_model(model, ctx_params)	基于模型创建推理上下文
llama_decode(ctx, batch)	执行一次前向传播（Prefill 或 Decode）
llama_get_embeddings_seq(ctx, seq_id)	提取序列级别的嵌入向量
llama_sampler_chain_init(params)	初始化采样器链
llama_sampler_sample(sampler, ctx, -1)	从最后一个位置采样下一个 token
llama_vocab_is_eog(vocab, token)	判断 token 是否为生成结束标记
llama_memory_clear(memory, data)	清空 KV Cache，防止上下文污染
llama_batch_init(n_tokens, embd, n_seq_max)	分配 batch 所需内存

API 函数	用途说明
llama_batch_free(batch)	释放 batch 内存

本文件由课程组制定，最终解释权归课程组所有。